

Informe técnico de mejoras de robustez en generación de reportes PDF

Proyecto: TCG Admin

Archivo modificado: monthlyReports.ts

Autor: Christian y Nayade

Fecha: 10-Marzo-2026

1. Contexto

Luego de corregir el error que impedía la generación de reportes PDF relacionado con la propiedad `finalY`, se realizó una revisión adicional del archivo `monthlyReports.ts` con el objetivo de mejorar la estabilidad del módulo y prevenir posibles errores futuros durante la generación de reportes.

Durante esta revisión se identificaron algunos escenarios donde el sistema podría fallar si los datos provenientes del sistema de analítica o de la base de datos no contienen información.

Estas mejoras no modifican la lógica funcional del sistema, pero permiten que el módulo de reportes sea más robusto frente a situaciones donde los datos estén incompletos o vacíos.

2. Problema identificado

El sistema genera múltiples tablas dentro del reporte PDF utilizando datos provenientes de estructuras como:

- `topStores`
- `topGames`
- `topActivities`
- `gameCategoryParticipation`

Estas estructuras se utilizan mediante operaciones como:

```
data.topStores.map(...)
```

El problema es que, si por alguna razón el servicio de analítica devuelve un valor `undefined`, `null` o una colección vacía, el método `map()` generaría un error en tiempo de ejecución.

Este tipo de error es común en aplicaciones que dependen de datos externos, ya que no siempre se puede garantizar que las colecciones contengan información. En ese escenario el sistema produciría errores como:

Cannot read properties of undefined (reading 'map')

lo que impediría continuar con la generación del reporte.

3. Solución implementada

Para evitar este tipo de errores se aplicó una estrategia de **programación defensiva**, que consiste en validar la existencia de los datos antes de iterarlos.

El código original utilizaba el siguiente patrón:

```
data.topStores.map((store, index) => ...)
```

Este patrón fue reemplazado por una versión segura:

```
(data.topStores || []).map((store, index) => ...)
```

Con esta modificación:

- Si topStores contiene datos, el sistema funciona normalmente.
- Si topStores es undefined o null, se utiliza un arreglo vacío.
- El método map() sigue ejecutándose sin provocar errores.

Este mismo ajuste se aplicó a otras estructuras utilizadas en la generación de tablas dentro del reporte.

4. Validación adicional del posicionamiento de tablas

Durante la generación del PDF se calcula la posición vertical donde deben comenzar las siguientes secciones del documento.

Este cálculo se realiza utilizando la propiedad:

```
doc.lastAutoTable.finalY
```

Sin embargo, en algunos escenarios excepcionales podría ocurrir que lastAutoTable no exista (por ejemplo si una tabla no fue generada previamente).

Para prevenir posibles errores se añadió una validación adicional antes de utilizar esta propiedad.

La implementación asegura que, si lastAutoTable no está disponible, el sistema utilizará una posición predeterminada para continuar generando el contenido del documento.

La implementación final quedó de la siguiente forma:

```
let nextY = (doc as any).lastAutoTable?.finalY  
  ? (doc as any).lastAutoTable.finalY + L.spacingAfterTable  
  : L.titleTop + 10;
```

De esta manera:

- Si existe una tabla previa, se utiliza su posición final como referencia.
- Si no existe, el sistema utiliza una posición segura basada en el layout del documento.

Esto evita errores relacionados con valores undefined y asegura que el contenido del reporte continúe generándose correctamente y se evita que el reporte falle en situaciones inesperadas.

5. Mejora en manejo de tablas sin datos

También se identificó que el sistema podía generar tablas vacías cuando no existían datos disponibles para ciertos rankings.

Para mejorar la experiencia del usuario se incorporó una validación que permite mostrar un mensaje informativo cuando no existen datos para una sección específica del reporte.

Por ejemplo:

- Cuando no existen datos de tiendas para el período seleccionado.
- Cuando no existen actividades registradas.
- Cuando no hay participación en ciertas categorías.

Esto permite que el reporte sea más claro y comprensible para los usuarios finales.

6. Resultado

Luego de implementar estas mejoras:

- El módulo de generación de reportes es más robusto.
- Se previenen errores asociados a datos inexistentes.
- El sistema puede generar reportes incluso cuando algunos conjuntos de datos están vacíos.
- La estabilidad del proceso de generación de documentos mejora significativamente.

Estas mejoras fortalecen la confiabilidad del sistema y reducen la probabilidad de fallos en producción.

7. Beneficio técnico

La incorporación de validaciones preventivas es una práctica recomendada en sistemas que dependen de múltiples fuentes de datos.

Este enfoque permite que el sistema continúe funcionando correctamente incluso cuando algunos datos no están disponibles, mejorando la estabilidad general del módulo de reportes.